Fondements informatiques I

Cours 6: Listes

Sorina Ionica sorina.ionica@uvsq.fr

Bilal FAYE bilal.faye@uvsq.fr

Les listes : une collection d'éléments

Un **tableau** est une structure de données qui regroupe plusieurs éléments, chacun occupant une position.

Exemple : les moyens de transport

Indice	0	1	2	3
Élément	bus	voiture	vélo	trottinette

À retenir

Chaque élément possède un **indice** qui indique sa position. En Python, le premier indice est **0**.

Les listes : une collection d'éléments

Un **tableau** est une structure de données qui regroupe plusieurs éléments, chacun occupant une position.

Exemple : les moyens de transport

Indice	0	1	2	3
Élément	bus	voiture	vélo	trottinette

À retenir

Chaque élément possède un **indice** qui indique sa position. En Python, le premier indice est **0**.

En Python, on utilise le type `list` pour représenter une liste. Les éléments sont placés entre crochets `[]` et séparés par des virgules.

Créer des listes

```
# Liste de capitales européennes
capitales = ["Paris", "Berlin", "Madrid"]
# Liste d'entiers
entiers = [5, 0, 17, 42, 23]
```

Créer des listes en Python

La construction des listes en Python est très flexible : une liste peut contenir des éléments de **types différents**.

Exemple : une liste mixte

```
mixte = [7, "tennis", True, 5.3]
```

Cette liste contient quatre valeurs de types différents :

- `7` : entier (`int`) ;
- `"tennis"` : chaîne de caractères (`str`) ;
- `True` : booléen (`bool`) ;
- `5.3` : nombre décimal (`float`).

À retenir

Une liste Python peut regrouper des valeurs de types différents.

Connaître la taille d'une liste

La fonction intégrée `len()` permet de connaître le **nombre d'éléments** d'une liste.

Exemple

```
liste = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print("La liste contient", len(liste), "éléments.")
```

Résultat

```
La liste contient 10 éléments.
```

Attention

La liste contient 10 éléments, mais son dernier indice est **9**, car les indices commencent à 0.

Modifier une liste

La taille d'une liste Python est **variable** : on peut ajouter ou supprimer des éléments après sa création.

Ajouter un élément avec append()

```
fruits = ["pomme", "banane"]
fruits.append("orange")
print(fruits)  # ['pomme', 'banane', 'orange']
```

Supprimer un élément avec remove()

```
fruits.remove("banane")
print(fruits)  # ['pomme', 'orange']
```

À retenir

- `append(element)` ajoute un élément à la fin.
- `remove(element)` supprime la première occurrence de cet élément.

Accéder aux éléments d'une liste

Chaque élément d'une liste possède un **indice** qui indique sa position.

- Le premier indice est 0.
- Pour une liste de longueur n, les indices vont de 0 à $n - 1$.
- Un indice doit être un nombre entier.

Exemple

```
transports = ["bus", "voiture", "velo", "trotinette"]

print(transports[0])   # bus
print(transports[2])   # velo
```

Attention

Un indice qui n'existe pas dans la liste provoque une erreur. Pour une liste de 4 éléments, `transports[4]` est invalide.

Accéder aux éléments depuis la fin

Python permet aussi d'utiliser des **indices négatifs** : on compte alors à partir de la fin de la liste.

Élément	bus	voiture	vélo	trottinette
Indice positif	0	1	2	3
Indice négatif	-4	-3	-2	-1

Exemple

```
transports = ["bus", "voiture", "vélo", "trottinette"]

print(transports[-1])   # trottinette
print(transports[-2])   # vélo
print(transports[0])    # bus
print(transports[-4])   # bus
```

Avantage

Avec −1 et −2, on accède au dernier et à l'avant-dernier élément sans connaître la longueur de la liste.

Extraire une partie d'une liste : les tranches

Une **tranche** permet d'extraire plusieurs éléments d'une liste.

Syntaxe

- `L[m:k]` : éléments d'indice `m` à `k-1`.
- `L[m:]` : de l'indice `m` jusqu'à la fin.
- `L[:k]` : du début jusqu'à l'indice `k-1`.

Exemple

```
nombres = [2, 3, 5, 7, 11, 13, 17, 19, 21, 23]

print(nombres[2:6])    # [5, 7, 11, 13]
print(nombres[-8:-4]) # [5, 7, 11, 13]
print(nombres[3:])     # [7, 11, 13, 17, 19, 21, 23]
print(nombres[:5])     # [2, 3, 5, 7, 11]
print(nombres[:-3])    # [2, 3, 5, 7, 11, 13, 17]
```

La borne de fin est **exclue** : dans `L[m:k]`, l'élément d'indice `k` n'est pas inclus.

Modifier les éléments d'une liste

Les listes sont des objets **mutables** : on peut modifier leur contenu après leur création. Pour remplacer un élément, on utilise son indice : `liste[i] = nouvelle_valeur`.

Exemple

```
capitales = ["Paris", "Berlin", "Madrid", "Athènes", "Zagreb"]
capitales[2] = "Dublin"
print(capitales)
# ['Paris', 'Berlin', 'Dublin', 'Athènes', 'Zagreb']
```

On peut aussi remplacer plusieurs éléments avec une tranche.

Remplacer une tranche

```
capitales[2:5] = ["Dublin", "Budapest", "Vienne"]
print(capitales)
# ['Paris', 'Berlin', 'Dublin', 'Budapest', 'Vienne']
```

Listes en compréhension : pourquoi ?

Objectif : créer une liste contenant les carrés des entiers de 0 à 9.

Méthode classique avec une boucle

```
carrees = []
for i in range(10):
    carrees.append(i**2)
print(carrees)
```

Listes en compréhension : pourquoi ?

Objectif : créer une liste contenant les carrés des entiers de 0 à 9.

Méthode classique avec une boucle

```
carrees = []
for i in range(10):
    carrees.append(i**2)
print(carrees)
```

Même résultat avec une compréhension

```
carrees = [i**2 for i in range(10)]
print(carrees)
```

À retenir

Une compréhension permet de créer une liste de façon **plus concise**, à partir d'une séquence de valeurs.

Comprendre la syntaxe

Syntaxe générale

```
[expression for variable in sequence]
```

- `sequence` : les valeurs parcourues, par exemple `range(10)`.
- `variable` : prend successivement chaque valeur.
- `expression` : calcule la valeur ajoutée à la nouvelle liste.

Exemple : les carrés des nombres de 0 à 9

```
carres = [x**2 for x in range(10)]
print(carres)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Comprendre la syntaxe

Syntaxe générale

```
[expression for variable in sequence]
```

- `sequence` : les valeurs parcourues, par exemple `range(10)`.
- `variable` : prend successivement chaque valeur.
- `expression` : calcule la valeur ajoutée à la nouvelle liste.

Exemple : les carrés des nombres de 0 à 9

```
carres = [x**2 for x in range(10)]
print(carres)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Exercice

Créer, avec une seule instruction, une liste contenant la table de multiplication de 5, de 5×1 à 5×10.

Tester l'appartenance à une liste

L'opérateur `in` permet de vérifier si un élément est présent dans une liste. Le résultat est un booléen : `True` ou `False`.

Exemple

```
nombres = [1, 3, 5, 7, 9, 11]
print(3 in nombres)  # True
print(2 in nombres)  # False
```

Tester l'appartenance à une liste

L'opérateur `in` permet de vérifier si un élément est présent dans une liste. Le résultat est un booléen : `True` ou `False`.

Exemple

```
nombres = [1, 3, 5, 7, 9, 11]
print(3 in nombres)  # True
print(2 in nombres)  # False
```

Combiner plusieurs tests

On peut utiliser `and`/`or` pour vérifier plusieurs conditions, et `not in` pour tester l'absence d'un élément.

```
print(5 in nombres and 7 in nombres)  # True
print(4 not in nombres)               # True
print(5 in nombres or 4 not in nombres)  # True
```

Tester l'appartenance à une liste

L'opérateur `in` permet de vérifier si un élément est présent dans une liste. Le résultat est un booléen : `True` ou `False`.

Exemple

```
nombres = [1, 3, 5, 7, 9, 11]
print(3 in nombres)  # True
print(2 in nombres)  # False
```

Combiner plusieurs tests

On peut utiliser `and`/`or` pour vérifier plusieurs conditions, et `not in` pour tester l'absence d'un élément.

```
print(5 in nombres and 7 in nombres)  # True
print(4 not in nombres)               # True
print(5 in nombres or 4 not in nombres)  # True
```

- `x in liste` : vrai si `x` est présent.
- `x not in liste` : vrai si `x` est absent.

Copier une liste : attention à =

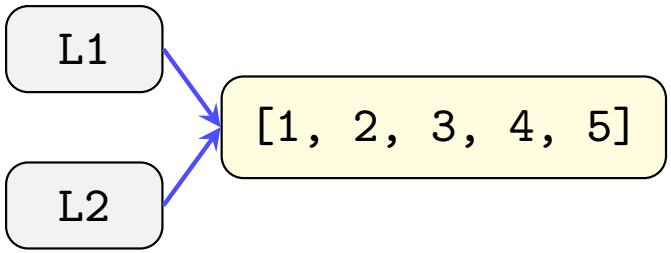
Qu'affiche ce programme ?

```
L1 = [1, 2, 3, 4]
L2 = L1
L2.append(5)
print(L1)
print(L2)
```

Copier une liste : attention à =

Qu'affiche ce programme ?

```
L1 = [1, 2, 3, 4]
L2 = L1
L2.append(5)
print(L1)
print(L2)
```



Même liste en mémoire

Résultat

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

L'idée essentielle

`L2 = L1` ne crée pas une nouvelle liste : les deux variables référencent **le même objet**. Modifier la liste via l'une modifie donc ce que l'on observe via l'autre.

Créer une copie avec une tranche

Pour obtenir deux listes indépendantes, on peut utiliser `L1[:]` : cette tranche reprend tous les éléments de `L1`.

Créer puis modifier la copie

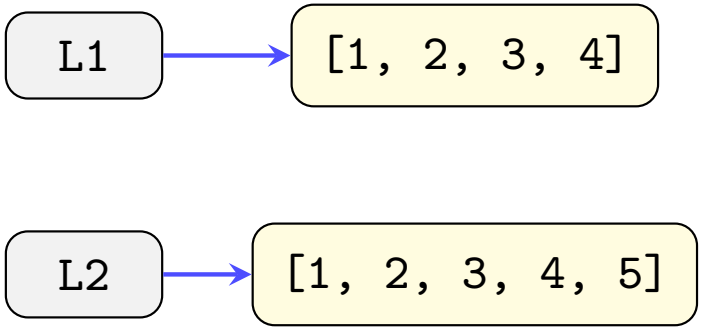
```
L1 = [1, 2, 3, 4]
L2 = L1[:]
L2.append(5)
print(L1)
print(L2)
```

Créer une copie avec une tranche

Pour obtenir deux listes indépendantes, on peut utiliser `L1[:]` : cette tranche reprend tous les éléments de `L1`.

Créer puis modifier la copie

```
L1 = [1, 2, 3, 4]
L2 = L1[:]
L2.append(5)
print(L1)
print(L2)
```



Deux listes distinctes

Résultat

```
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
```

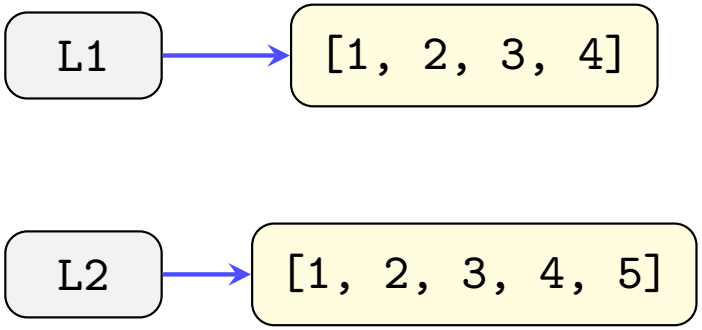
`L2 = L1[:]` crée une nouvelle liste contenant les mêmes éléments. Ajouter ou supprimer un élément de `L2` ne modifie pas `L1`.

Une autre méthode : `list()`

La fonction `list()` permet également de créer une copie superficielle d'une liste.

Exemple

```
L1 = [1, 2, 3, 4]
L2 = list(L1)
L2.append(5)
print(L1)
print(L2)
```



Résultat

```
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
```

Deux façons de copier

`L2 = L1[:]` ou `L2 = list(L1)`

Concaténer deux listes

L'opérateur + permet de **rassembler deux listes** pour former une nouvelle liste.

Que va afficher ce programme ?

```
capitales1 = ["Paris", "Berlin", "Madrid"]
capitales2 = ["Bruxelles", "Oslo"]

capitales1 + capitales2
print(capitales1)
```

Concaténer deux listes

L'opérateur + permet de **rassembler deux listes** pour former une nouvelle liste.

Que va afficher ce programme ?

```
capitales1 = ["Paris", "Berlin", "Madrid"]
capitales2 = ["Bruxelles", "Oslo"]

capitales1 + capitales2
print(capitales1)
```

Attention !

L'expression `capitales1 + capitales2` construit une nouvelle liste, mais ne modifie pas `capitales1` si le résultat n'est pas conservé.

Concaténer deux listes

L'opérateur + permet de **rassembler deux listes** pour former une nouvelle liste.

Que va afficher ce programme ?

```
capitales1 = ["Paris", "Berlin", "Madrid"]
capitales2 = ["Bruxelles", "Oslo"]

capitales1 + capitales2
print(capitales1)
```

Attention !

L'expression `capitales1 + capitales2` construit une nouvelle liste, mais ne modifie pas `capitales1` si le résultat n'est pas conservé.

Créer une nouvelle liste

```
capitales3 = capitales1 + capitales2
print(capitales3)
# ['Paris', 'Berlin', 'Madrid', 'Bruxelles', 'Oslo']
```

Ajouter les éléments d'une liste à une autre

Si l'objectif est de modifier une liste existante, on peut utiliser `extend()`.

Exemple

```
capitales1 = ["Paris", "Berlin", "Madrid"]
capitales2 = ["Bruxelles", "Oslo"]

capitales1.extend(capitales2)
print(capitales1)
# ['Paris', 'Berlin', 'Madrid', 'Bruxelles', 'Oslo']
```

À retenir

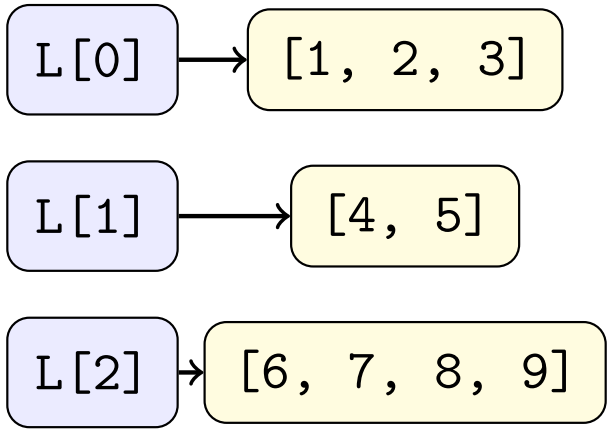
- `L3 = L1 + L2` : crée une nouvelle liste.
- `L1.extend(L2)` : ajoute les éléments de `L2` à `L1`.

Listes imbriquées : accéder aux éléments

Une **liste imbriquée** est une liste dont certains éléments sont eux-mêmes des listes.

Exemple

```
L = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
```

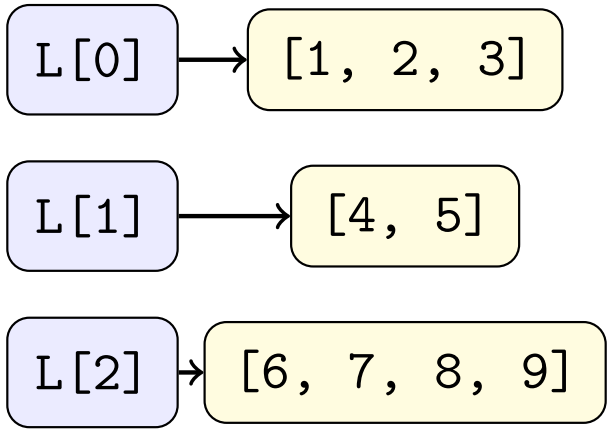


Listes imbriquées : accéder aux éléments

Une **liste imbriquée** est une liste dont certains éléments sont eux-mêmes des listes.

Exemple

```
L = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
```



Accéder à une liste ou à un élément

```
print(L[0])      # [1, 2, 3]
print(L[1])      # [4, 5]
print(L[2][1])   # 7
```

Parcourir une liste imbriquée

On veut calculer la somme de tous les nombres contenus dans `L`.

Deux boucles imbriquées

```
L = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
somme = 0
for sous_liste in L:
    for nombre in sous_liste:
        somme += nombre
print(somme)  # 45
```

Parcourir une liste imbriquée

On veut calculer la somme de tous les nombres contenus dans `L`.

Deux boucles imbriquées

```
L = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
somme = 0
for sous_liste in L:
    for nombre in sous_liste:
        somme += nombre
print(somme)  # 45
```

Comment fonctionne le parcours ?

1. La première boucle parcourt chaque sous-liste.
2. La seconde boucle parcourt chaque nombre de cette sous-liste.
3. Chaque nombre est ajouté à `somme`.

Parcourir une liste imbriquée

On veut calculer la somme de tous les nombres contenus dans `L`.

Deux boucles imbriquées

```
L = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
somme = 0
for sous_liste in L:
    for nombre in sous_liste:
        somme += nombre
print(somme)  # 45
```

Comment fonctionne le parcours ?

1. La première boucle parcourt chaque sous-liste.
2. La seconde boucle parcourt chaque nombre de cette sous-liste.
3. Chaque nombre est ajouté à `somme`.

La liste contient trois sous-listes, mais elles n'ont pas toutes la même longueur. Les boucles `for` parcourent chaque élément sans avoir besoin de connaître ces longueurs à l'avance.

Créer une liste de listes : attention aux références

On veut créer une liste de m lignes contenant chacune n zéros.

Une première tentative

```
m, n = 3, 4
L = [[0] * n] * m
```

Cette instruction crée bien m références vers des lignes, mais **toutes désignent la même liste**.

Modifier un élément

```
L[0][1] = 1
```

Créer une liste de listes : attention aux références

On veut créer une liste de m lignes contenant chacune n zéros.

Une première tentative

```
m, n = 3, 4
L = [[0] * n] * m
```

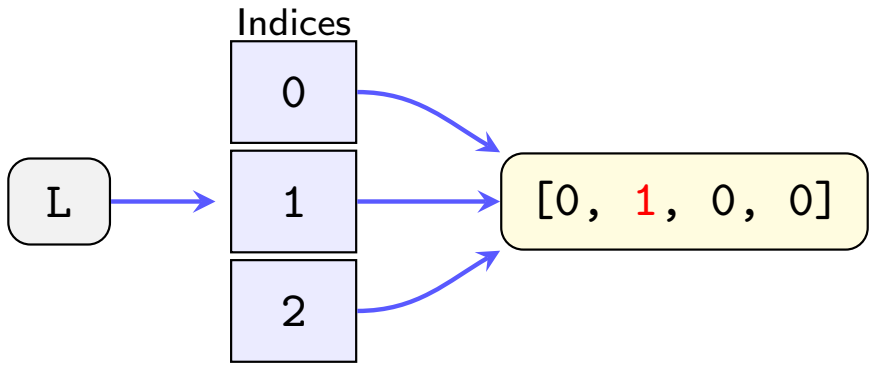
Cette instruction crée bien m références vers des lignes, mais **toutes désignent la même liste**.

Modifier un élément

```
L[0][1] = 1
```

Résultat

```
[[0, 1, 0, 0],
 [0, 1, 0, 0],
 [0, 1, 0, 0]]
```



Les trois cases référencent
la même sous-liste.

La bonne méthode : créer chaque ligne séparément

Une compréhension de liste permet de créer une nouvelle ligne à chaque itération.

Créer une grille de zéros

```
m, n = 3, 4
L = [[0] * n for i in range(m)]
```

Modifier un élément

```
L[0][1] = 1
print(L)
```

La bonne méthode : créer chaque ligne séparément

Une compréhension de liste permet de créer une nouvelle ligne à chaque itération.

Créer une grille de zéros

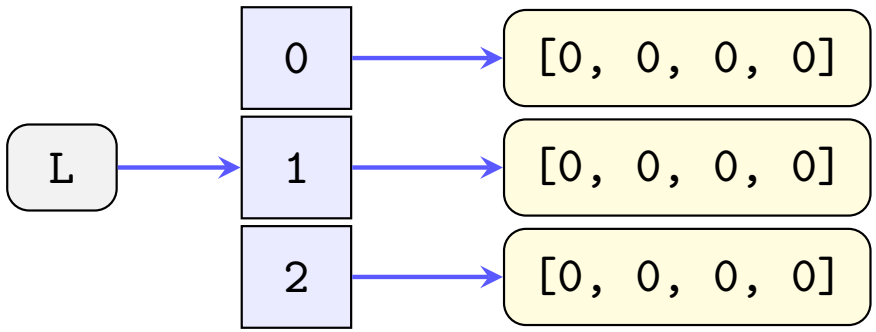
```
m, n = 3, 4
L = [[0] * n for i in range(m)]
```

Modifier un élément

```
L[0][1] = 1
print(L)
```

Résultat

```
[[0, 1, 0, 0],
 [0, 0, 0, 0],
 [0, 0, 0, 0]]
```



Chaque position désigne une ligne indépendante.

Exercice : construire une matrice

Écrire une fonction `construire_matrice(n)` qui construit une matrice carrée de taille n suivant la règle :

$$L_{i,j} = \begin{cases} 1 & \text{si } i = j, \\ 2 & \text{si } i < j, \\ 0 & \text{si } i > j. \end{cases}$$

Pour $n = 4$, on veut obtenir :

$$\begin{bmatrix} 1 & 2 & 2 & 2 \\ 0 & 1 & 2 & 2 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Consignes

- Créer d'abord une matrice remplie de zéros.
- Compléter les éléments en parcourant les lignes et les colonnes.
- La fonction doit **renvoyer** la matrice avec `return`.

Correction : construire une matrice

Une solution avec deux boucles

```
def construire_matrice(n):
    L = [[0] * n for i in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                L[i][j] = 1
            elif i < j:
                L[i][j] = 2
    return L
print(construire_matrice(4))
```

Correction : construire une matrice

Une solution avec deux boucles

```
def construire_matrice(n):
    L = [[0] * n for i in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                L[i][j] = 1
            elif i < j:
                L[i][j] = 2
    return L
print(construire_matrice(4))
```

Pourquoi cette solution fonctionne-t-elle ?

- La compréhension crée n lignes indépendantes.
- Les deux boucles parcourent chaque position (i, j).
- Les conditions déterminent la valeur à placer.
- `return L` renvoie la matrice construite.

Une autre solution : construire chaque ligne

On peut aussi construire chaque ligne directement, sans modifier chaque élément séparément.

Correction compacte

```
def construire_matrice(n):
    L = []
    for i in range(n):
        ligne = [0] * i + [1] + [2] * (n - i - 1)
        L.append(ligne)
    return L
print(construire_matrice(4))
```

Une autre solution : construire chaque ligne

On peut aussi construire chaque ligne directement, sans modifier chaque élément séparément.

Correction compacte

```
def construire_matrice(n):
    L = []
    for i in range(n):
        ligne = [0] * i + [1] + [2] * (n - i - 1)
        L.append(ligne)
    return L
print(construire_matrice(4))
```

Comprendre la construction d'une ligne

Pour la ligne d'indice `i`, on assemble :

- `i` zéros ;
- un `1` sur la diagonale ;
- `n - i - 1` deux après la diagonale.